C L I P M I N D A I

Milestone 2 Report

Transcript Generation and AI Summarization

Video Summarization & Key Moments Detection Platform

Project	**ClipMind AI: AI-powered video transcription, summarization and key moments platform**
Report	Milestone 2 of 4 (Weeks 3 and 4)
Submitted by	Team 1
Domain	Artificial Intelligence
Document type	Project documentation (milestone report)

From uploaded videos to a stored, timestamped transcript and a BART-generated short and detailed summary generation.

Contents

1. Milestone at a Glance
2. Timeline
3. Concepts in Short
4. System Design
5. Implementation
6. API Reference
7. Testing and Results
8. Challenges and Resolutions
9. Evaluation Against Milestone Criteria
10. Conclusion and Next Steps
11. Appendix: Screenshots

1. Milestone at a Glance

Milestone 2 added the first AI features. A processed video is now converted into a timestamped text transcript by Whisper, the transcript is stored in PostgreSQL, and a BART summarization model turns it into a short summary and a detailed summary. Statuses, retry and duplicate-job protection make the process safe to use.

Table 1: Summary of Milestone 2

Item	Details
Duration	Weeks 3 and 4
Focus	Transcript generation and AI summarization
Main deliverables	Transcript and Summary database models, Alembic migration, Pydantic schemas, FFmpeg audio extraction, local Whisper transcription, transcript storage and status, BART hierarchical summarization, summary quality checks, duplicate-job protection, retry, transcript and summary APIs, frontend Transcript and Summary pages
Technology used	OpenAI Whisper (base, local), FFmpeg, Hugging Face Transformers with facebook/bart-large-cnn, PyTorch, PostgreSQL, SQLAlchemy, Alembic, FastAPI, React
Tests	22 passing tests for the transcript and summarization backend; 66+ passing tests in the full repository suite at milestone review
Outcome	Working transcript and summary pipeline; summary faithfulness remains an area for continued improvement (see Challenges)

1.1 Objectives

- Extract audio from the uploaded video and convert speech to text with Whisper.
- Store the transcript, with timestamps, in the database against the correct video and user.
- Generate a short summary and a detailed summary from the stored transcript, including very long transcripts.
- Show clear statuses (not started, processing, completed, failed) so the screen is never blank.
- Prevent duplicate processing and keep the transcript safe if summarization fails.

2. Timeline



Figure 1: Eight-week project timeline with Milestone 2 highlighted

Table 2: Week-by-week activities in Milestone 2

Week	Activities	Result
Week 3	Agreed the data contract: User, Video, Transcript, Summary relationships and one set of statuses. Confirmed PostgreSQL with SQLAlchemy as the only database approach. Built the FFmpeg audio-extraction service (mono, 16 kHz WAV). Integrated local Whisper base and stored text, language and timestamped segments. Fixed the SQLAlchemy model-registry error caused by the missing Summary model.	Transcription working; models agreed
Week 4	Merged the Alembic migration that adds transcript language and segments and creates the summaries table. Added Pydantic schemas and the transcript and summary APIs with ownership checks. Built the summarization service: token-aware chunking, hierarchical BART summaries, quality checks. Added status updates, retry and duplicate-job protection. Built the Transcript and Summary pages and fixed frontend and backend mismatches. Ran the test suite and end-to-end checks.	Short and detailed summaries stored and displayed

3. Concepts in Short

Table 3: Key concepts used in this milestone

Concept	Plain explanation	Where it is used
Speech-to-text	Turns spoken audio into written text.	Whisper transcription
Whisper	OpenAI's speech-recognition model. We run the base size locally, so no external API is called.	Transcript generation
Timestamped segments	Whisper returns the text in short pieces, each with a start and end time in seconds.	Stored as JSON; reused by key moments in Milestone 3

Concept	Plain explanation	Where it is used
Token and tokenizer	Models read text as tokens (word pieces). The tokenizer converts text to tokens and counts them.	Deciding how much text fits in one summarization step
Chunking	Splitting a long text into smaller parts that fit the model's limit.	Long transcripts
Abstractive summarization	The model writes new sentences instead of copying existing ones.	BART summary
BART	A sequence-to-sequence Transformer model; bart-large-cnn is trained for news-style summarization and accepts about 1,024 tokens.	Summary generation
Hierarchical summarization	Summarize each chunk, then summarize the summaries until the result fits in one step.	Handling a 1-2 hour lecture
State machine	A fixed set of statuses and allowed moves between them.	PENDING, PROCESSING, COMPLETED, FAILED
Alembic migration	A versioned script that updates the database structure safely.	Adding language, segments and the summaries table

4. System Design

4.1 Processing pipeline

The transcript is created once and reused. The summary reads the stored transcript and never runs speech-to-text again. This keeps the architecture efficient and lets later features (key moments, analytics) reuse the same transcript.

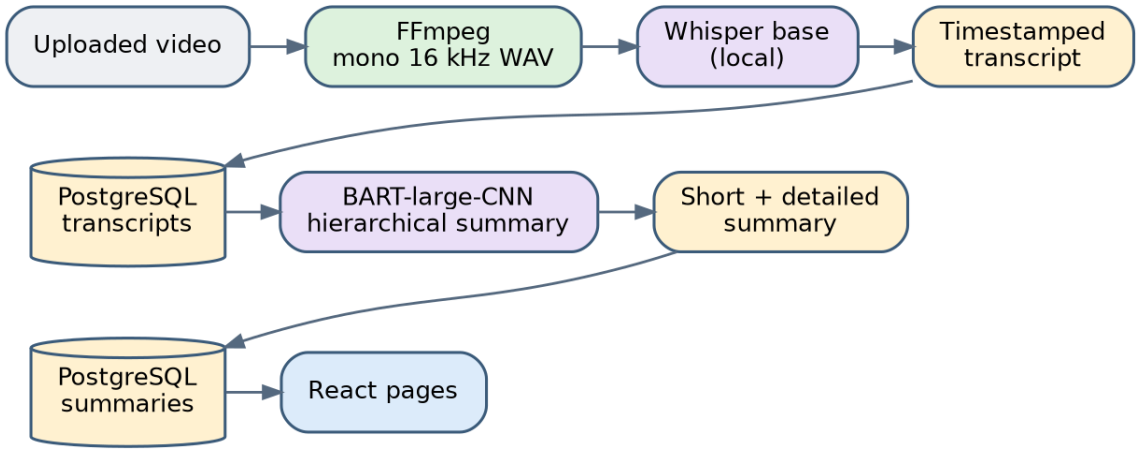


Figure 2: Transcript and summary pipeline

4.2 Database design

Each video has one transcript and each transcript has one summary. The ownership chain is User, then Video, then Transcript, then Summary, so a transcript can always be traced back to the user who owns it.

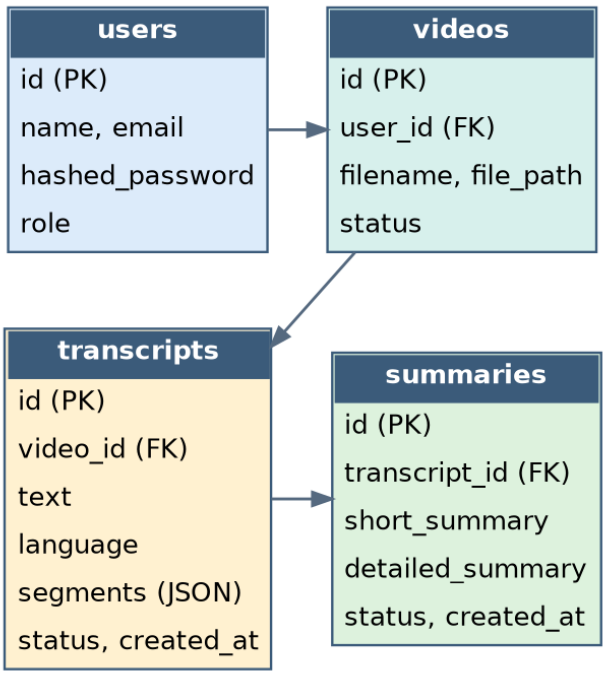


Figure 3: Entity relationships after Milestone 2

Table 4: New and changed tables

Table	Fields	Notes
transcripts	id, video_id (FK), text, language, segments (JSON), status, created_at	language (String 10) holds the language detected by Whisper; segments keeps Whisper's timestamped output
summaries	id, transcript_id (FK), short_summary, detailed_summary, status, created_at	Stored separately from the transcript so it can be regenerated without losing the transcript

The migration (revision 61b0338b2c08) was merged into main through a Pull Request. It adds the two transcript columns, creates the summaries table, defines the status values and makes sure the Alembic environment imports every model.

4.3 Status model

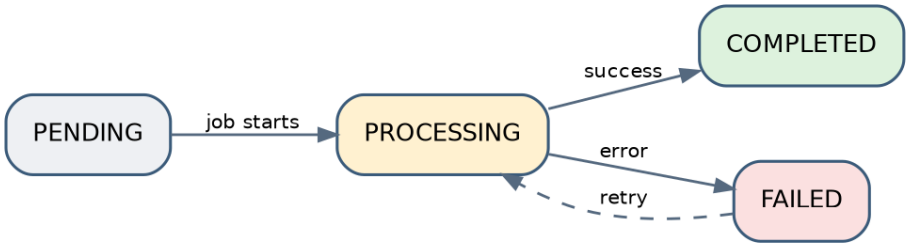


Figure 4: Transcript and summary statuses

Table 5: Meaning of each status

Status	Meaning	What the frontend shows
PENDING	Created but processing has not started	"Not generated yet" with a Generate button
PROCESSING	The result is being generated	Loading message such as "Generating summary..."
COMPLETED	The result is ready	The content and enabled action buttons
FAILED	Something went wrong	A clear error message and a Retry button

5. Implementation

5.1 Audio extraction with FFmpeg

Uploaded videos can have different containers, codecs, sample rates and channel counts. FFmpeg converts the audio to one format that Whisper handles reliably.

Table 6: FFmpeg audio settings

Setting	Value	Reason
Streams	Video removed (-vn)	Only audio is needed
Channels	1 (mono)	Speech recognition does not need stereo
Sample rate	16,000 Hz	The rate Whisper expects
Codec and container	PCM 16-bit WAV	Uncompressed and widely supported
Cleanup	Temporary WAV deleted after transcription	Saves storage

5.2 Transcription with Whisper

- Whisper runs **locally** in the backend; the model name comes from the WHISPER_MODEL setting and defaults to base.
- The model is loaded only when first needed and then cached, so the server starts quickly.
- Whisper returns the full text, the detected language and a list of segments with start time, end time and text.

- All of these are saved in PostgreSQL against the video, so opening the page never re-runs transcription.
- Failures are reported with short safe codes (audio file missing, model could not load, transcription failed). Technical details stay in the server log.

5.3 Summarization with BART

A two-hour lecture is far too long for BART's input limit, so the summarizer uses a hierarchical (map-reduce) method.

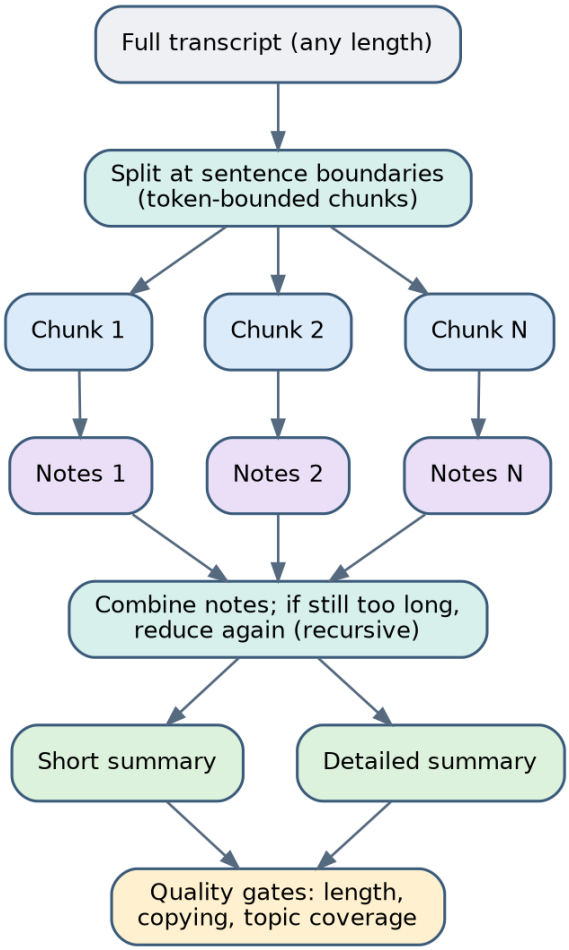


Figure 5: Hierarchical summarization of a long transcript

1. **Split:** the transcript is cut at sentence boundaries into chunks that fit a token budget (at most 512 tokens per first-pass chunk; overall input never exceeds 900 tokens). A sentence longer than the budget is split at word boundaries.
2. **Summarize each chunk** into short notes.
3. **Combine and reduce:** the notes are joined; if they are still too long they are summarized again, repeatedly, until they fit in one step.
4. **Final synthesis:** two outputs are generated from the combined notes, a short overview and a fuller detailed summary, each with its own length limits.
5. **Quality check:** the result is validated before it is saved (see below).

Table 7: Generation settings used with BART

Setting	Value	Purpose
Model	facebook/bart-large-cnn (LOCAL_SUMMARY_MODEL can change it)	Local Hugging Face model on CPU
Beam search	4 beams, length penalty 2.0, early stopping	More coherent output
Repetition control	no_repeat_ngram_size 3, repetition penalty 1.05, encoder_no_repeat_ngram_size 3	Avoid repeated phrases and long copied phrases
Token safety	Input checked before generation; error raised rather than silently truncating	Never lose the end of a transcript unnoticed
Concurrency	A lock around generation	Only one heavy generation at a time on the server

5.4 Summary quality checks

After generation, the service measures the summary and rejects output that is clearly poor. A rejected summary marks the job FAILED, and the transcript is kept.

Table 8: Quality gates

Check	Rule	Why
Empty output	Short or detailed summary must not be empty	Catch model failures
Minimum length	Scaled to transcript size (for example at least 25 and 30 words for transcripts of 700+ words); relaxed for very short transcripts	Avoid one-line summaries of long videos
Copying	Reject if 40 or more consecutive words are copied from the transcript	A summary should paraphrase
Extractive style	Reject if 14+ consecutive words are shared and most sentences are exact copies	Detect copy-paste output
Topic coverage	Reject if the 30 most frequent terms are mostly missing and the text is also extractive	Detect off-topic output

5.5 Duplicate processing and failure handling

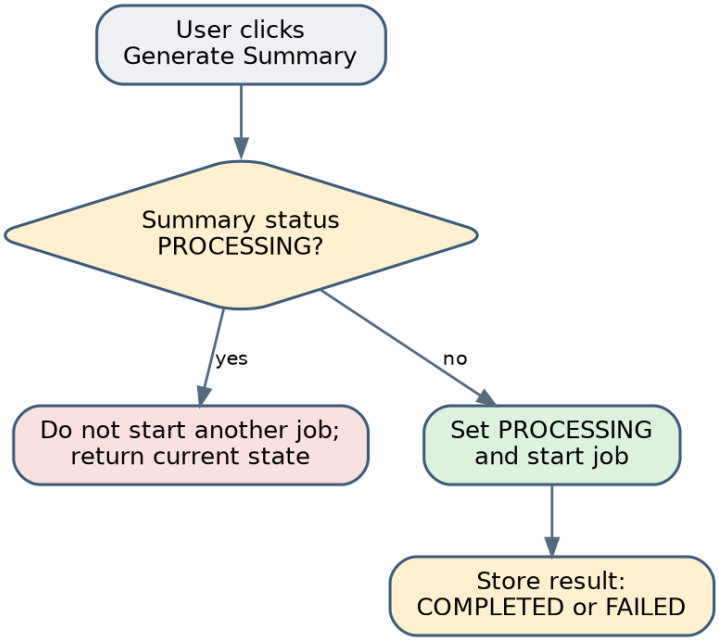


Figure 6: Preventing duplicate summary jobs

- Clicking Generate Summary several times starts only one job; the backend checks the status first and the button is disabled while processing.
- If summarization fails, the saved transcript is **not lost**; the user can read it and retry the summary.
- If Whisper or audio extraction fails, the transcript is marked FAILED and the error is shown with a retry option.
- An early placeholder summarizer (first sentences of the transcript) was used to validate the pipeline before BART was integrated.

5.6 Frontend pages

- **Transcript page:** video title, status, the generated transcript with timestamps, keyword search, edit, download and a Generate Summary action. Failed or empty transcripts show a message instead of a blank page.
- **Summary page:** short and detailed summaries in separate sections, status, Generate or Regenerate and Retry buttons, loading indicator, error message and last generated time.
- A stale-response test makes sure an old response for Video A never overwrites the page after the user switches to Video B.

6. API Reference

Table 9: Transcript and summary endpoints

Method	Endpoint	Purpose
POST	/videos/{video_id}/transcript	Start transcript processing
GET	/videos/{video_id}/transcript	Return the transcript, language, segments and status
GET	/videos/{video_id}/transcript/download	Download the completed transcript
GET / POST	Summary endpoints under /videos/{video_id}/...	Get the summary, generate it, or retry after failure

All endpoints require a valid JWT. The backend checks the chain User, Video, Transcript, Summary so that a user can only read data for videos they own.

7. Testing and Results

Table 10: Testing activities and outcomes

Area	What was checked	Outcome
Model registry	Repository tests after removing temporary dummy models	Failed first (Summary model missing); passed after the Summary model and migration were merged
Migration	Revision 61b0338b2c08 adds language, segments and the summaries table; all models are imported by Alembic	Merged and in use
Module tests	Transcript and summarization backend	22 tests passed
Full suite	All backend tests in the repository	66+ tests passed at milestone review
Ownership	User cannot read another user's transcript or summary	Enforced in the API
End-to-end run	Upload, transcript and summary on real videos	Pipeline worked; one summary changed the meaning of a sentence, logged as a quality issue
Processing time and accuracy	Whisper base and BART on test videos	

8. Challenges and Resolutions

Table 11: Problems faced and how they were handled

Problem	Cause	Resolution	Status
SQLAlchemy mapper error	Video pointed to Transcript, which needed a Summary model that was not yet pushed	Summary model added and merged; tests rerun without dummy models	Resolved
Model contract mismatch	Transcript lacked language and segments; status names differed from the agreed set; schemas and migrations were missing	Agreed contract (PENDING, PROCESSING, COMPLETED, FAILED), schemas added, Alembic migration merged	Resolved
Conflicting database plan	A MongoDB TODO remained in the video router while the implementation used PostgreSQL	Decision: PostgreSQL with SQLAlchemy only; no duplicate models or tables	Resolved
Transcripts too long for BART	BART accepts about 1,024 tokens	Token-aware chunking and hierarchical summarization	Resolved
Duplicate jobs	Repeated clicks could start several summaries	Status check before starting, disabled button	Resolved
Summary changed a sentence's meaning	Abstractive models can paraphrase incorrectly	Quality gates added; stricter faithfulness validation (comparing meaning, not only length and keywords) planned	Open
Generation error mentioning encoder_no_repeat_ngram_size	Reported on one machine during processing	The parameter is now set explicitly in the generation call	
Registration 422 and other contract mismatches	Frontend and backend used different field names	Frontend payloads aligned with backend schemas	Resolved

9. Evaluation Against Milestone Criteria

Table 12: Milestone 2 evaluation checklist

Criterion	Status
Transcript generation and summarization workflows implemented	Done
Whisper integration functional	Done
AI summary generation working	Done
Transcript storage and management completed	Done
Summary quality evaluated	In progress (faithfulness validation to improve)

10. Conclusion and Next Steps

Milestone 2 gave ClipMind AI its core intelligence. One reusable timestamped transcript is created and stored, and from it a BART pipeline produces short and detailed summaries with quality checks, status tracking and retry.

Next (Milestone 3): reuse the stored timestamps to detect topics and key moments with Sentence Transformer embeddings, create highlight clips, and build the analytics dashboards.

11. Appendix: Screenshots

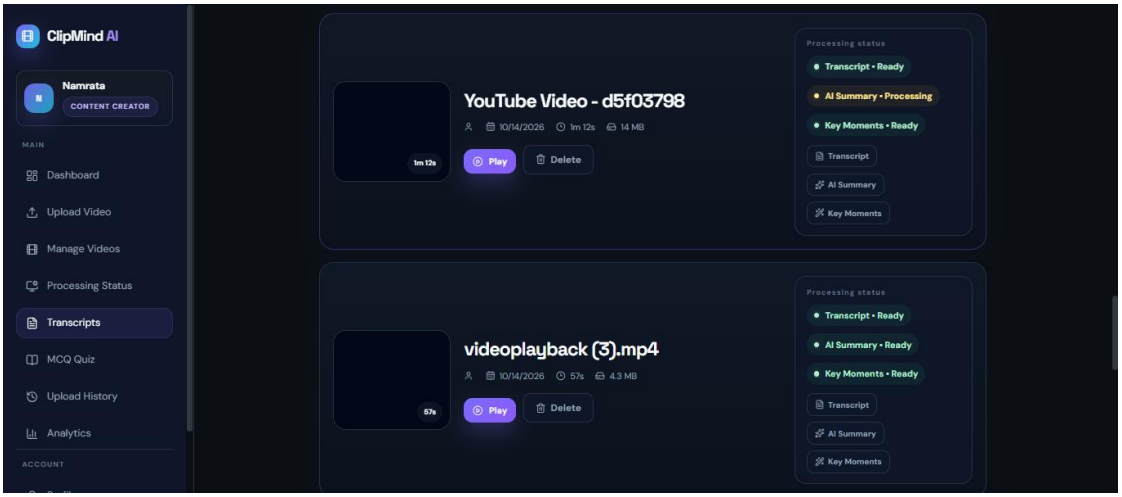


Figure 7: Transcript page showing status, timestamps and size

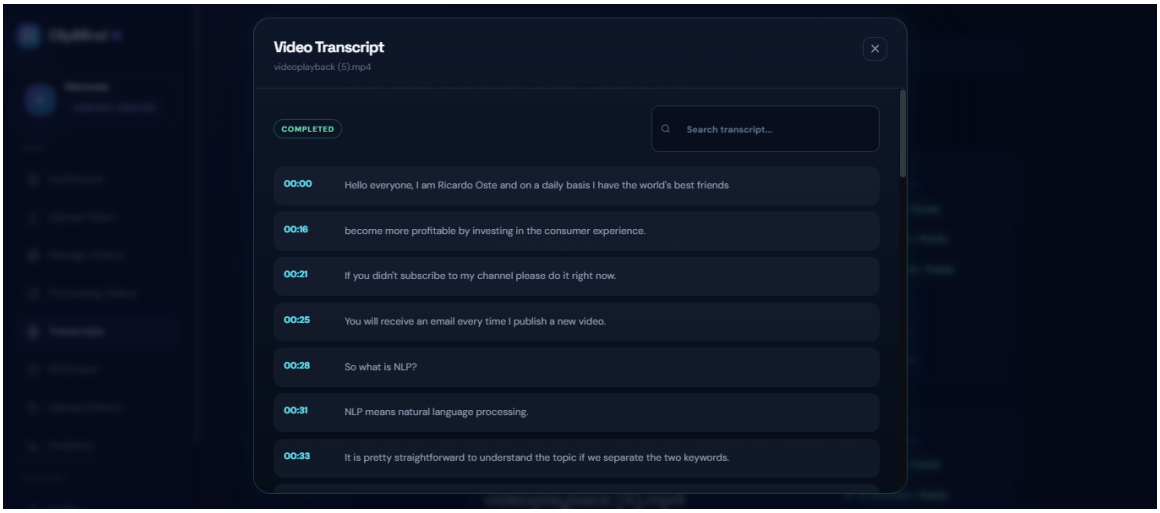
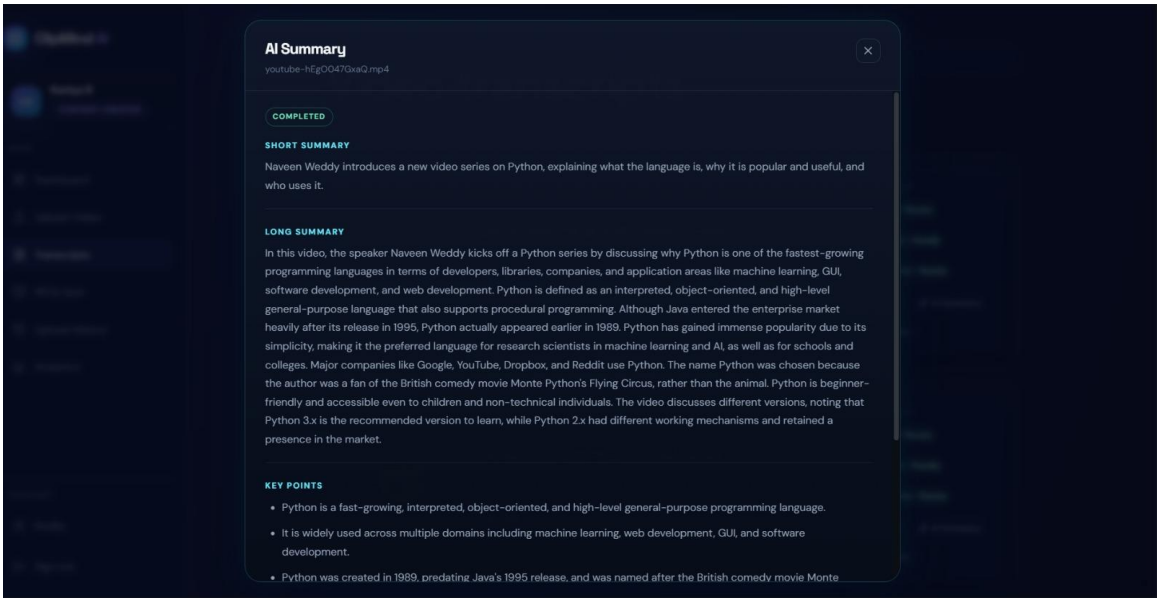


Figure 8: Transcripts with timestamps



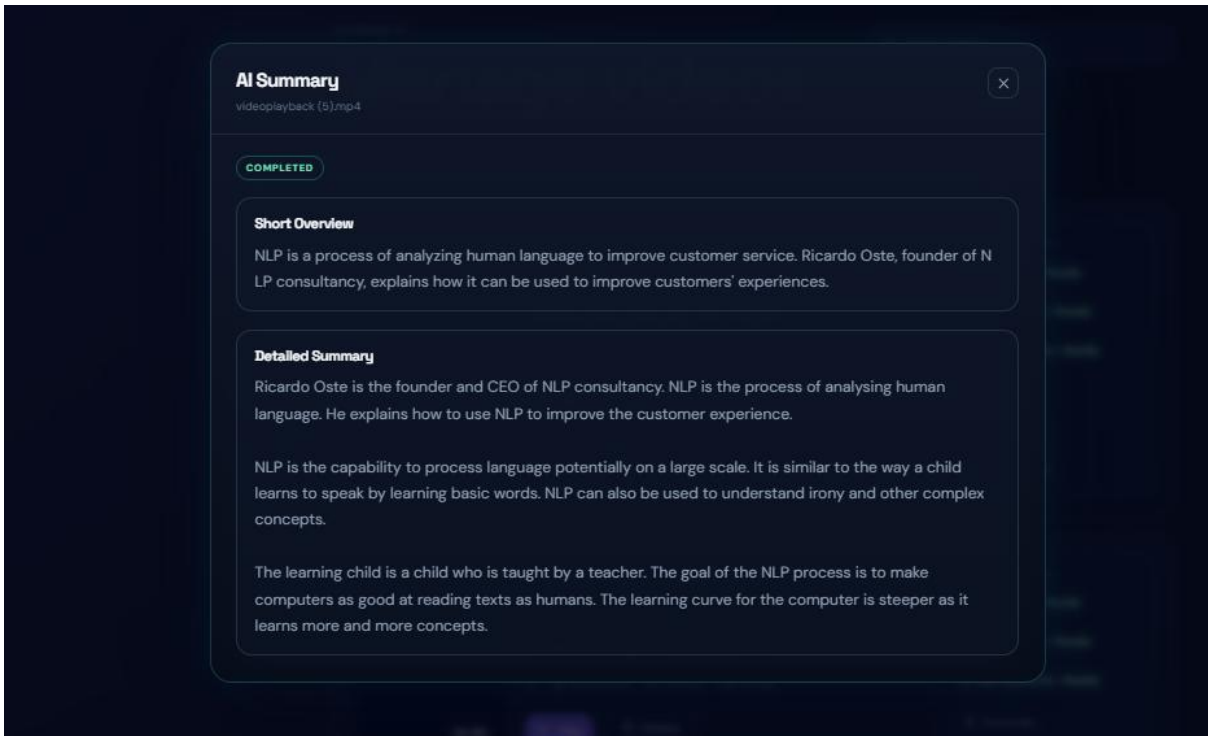


Figure 9: Short and detailed summaries